

GCI

Unsupervised Learning



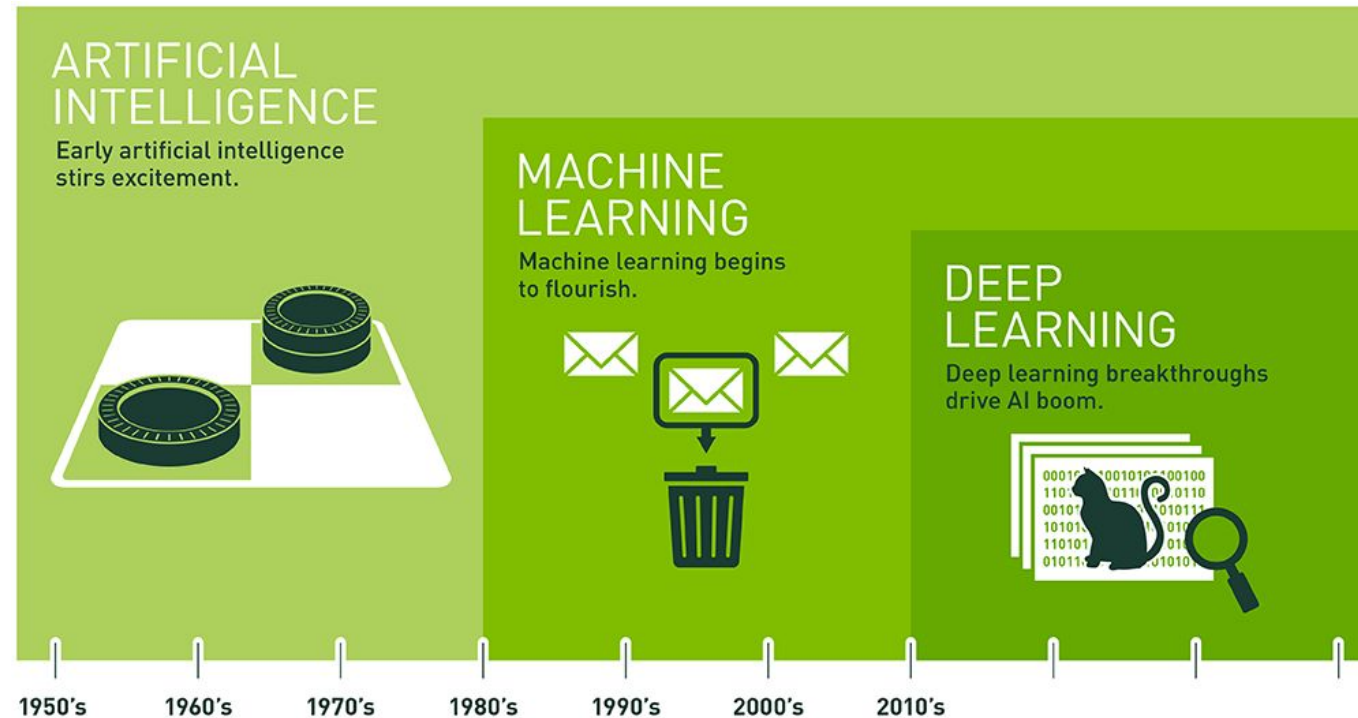
- What is Unsupervised Learning?
- Clustering
- Dimensionality Reduction

What is Unsupervised Learning?

AI? Machine Learning? Deep Learning?



- **Artificial Intelligence (AI)**: A Research field that aims to create an artificial human-level intelligence
- **Machine Learning**: Method to train a program to learn patterns from data
- **Deep Learning**: Method of machine learning that uses neural networks



Reference:

<https://blogs.nvidia.com/blog/2016/07/29/whats-difference-artificial-intelligence-machine-learning-deep-learning-ai/>

What is Machine Learning?

- Machine Learning: Method to train a program to learn patterns from data
- By learning, the program can **make predictions from the learned patterns**

Loading Data



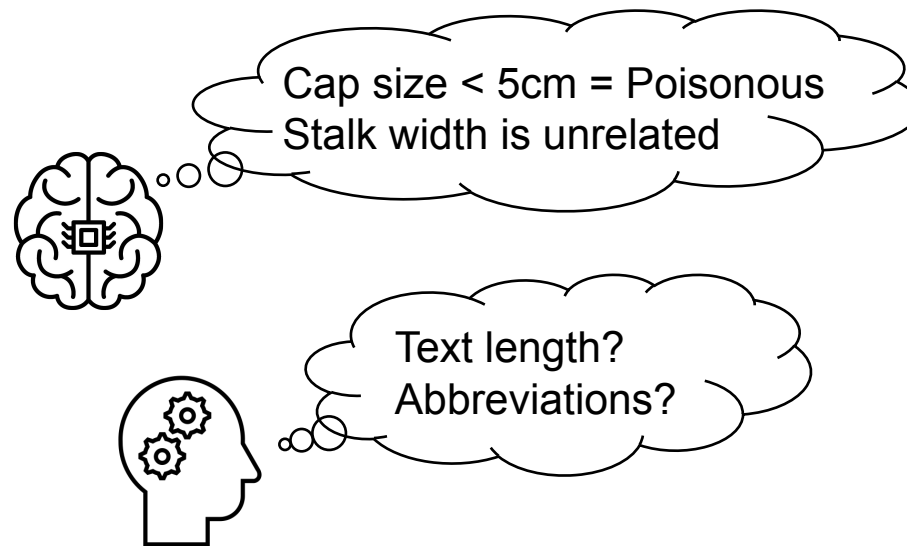
Cap size 10cm 5cm 7cm 4cm

Stalk width 1cm 2cm 5cm 2cm

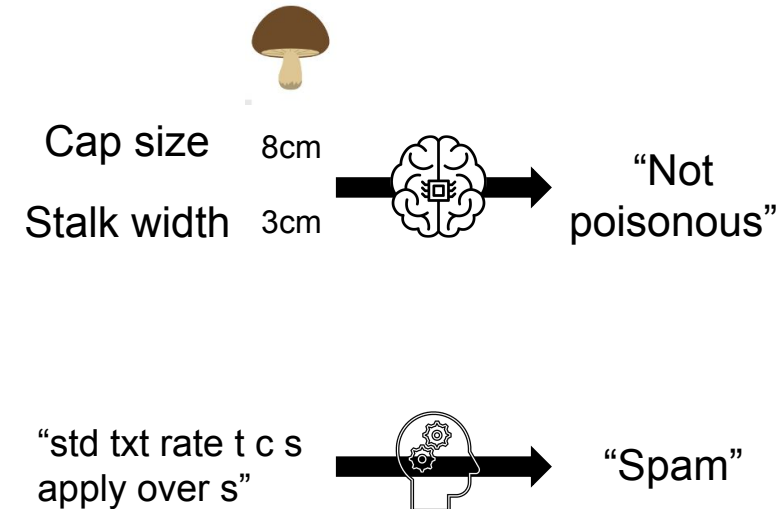
Poisonous? No Yes No Yes

“Sorry, I was just joking...” Not spam
“Free entry in 2 a wkly comp to win FA Cup final tkts” Spam
“Nah I don't think he goes to usf” Not spam

Learn from Data



Making Predictions



Textbooks, Past exams



How do I solve these problems?

Study for exam

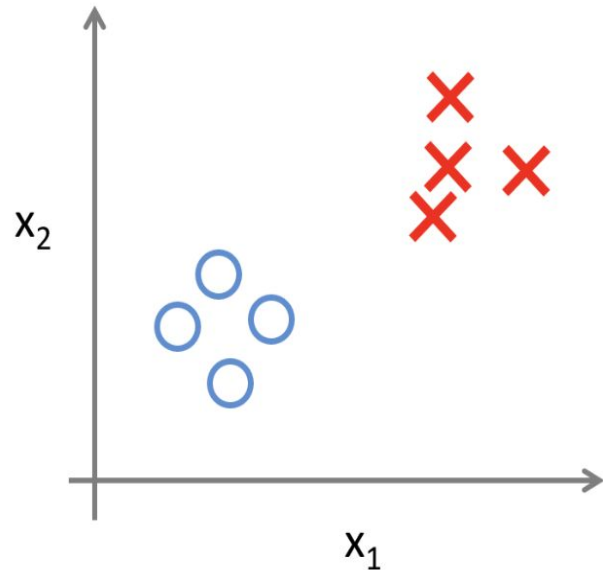


Actual exam

Types of Machine Learning

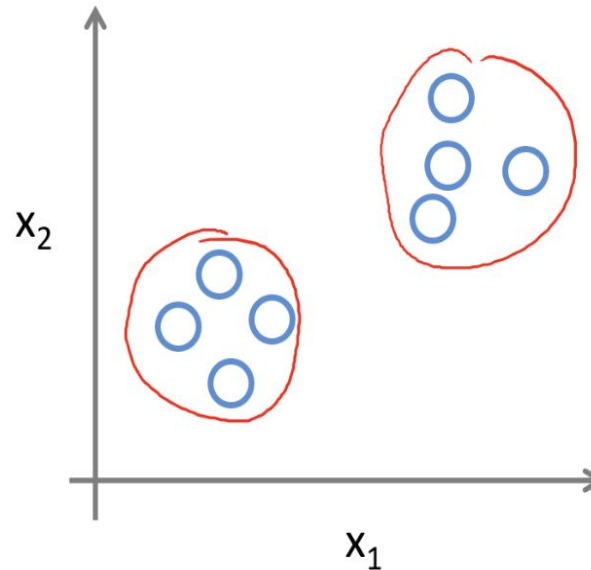


Supervised Learning



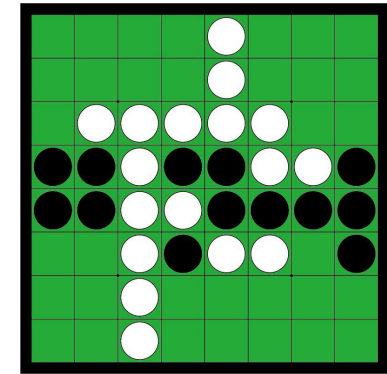
- Have ground-truth values
- Learn the relationship between input and output

Unsupervised Learning



- No ground-truth values
- Find patterns in the data

Reinforcement Learning



- Learn to maximize reward

Reference:
<https://lakshaysuri.wordpress.com/2017/03/19/machine-learning-supervised-vs-unsupervised-learning/>

What is Unsupervised Learning?

- Motivation: Discover patterns in the data
- Differences between supervised learning
 - **No target values to predict**
- Types of unsupervised learning
 - Clustering
 - Dimensionality Reduction
 - etc.

```
df = pd.read_csv('gci.csv')

X = df[['column1', 'column2', 'column3']]
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0
)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Supervised learning

```
df = pd.read_csv('gci.csv')

X = df[['column1', 'column2', 'column3']]

model = KMeans(n_clusters=5, random_state=0)
model.fit(X)

df['cluster'] = model.labels_
df.groupby('cluster').mean().plot(kind='bar')
```

Unsupervised learning

Clustering

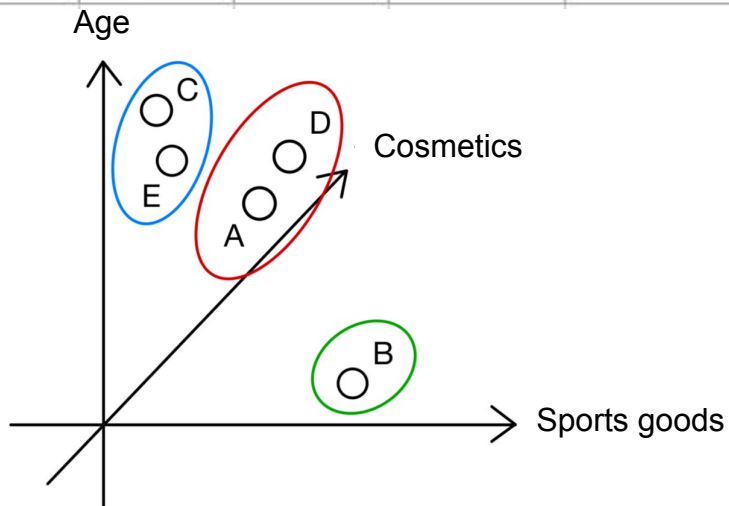
Clustering



- Find similar patterns in the data and group them as **clusters**
- e.g.

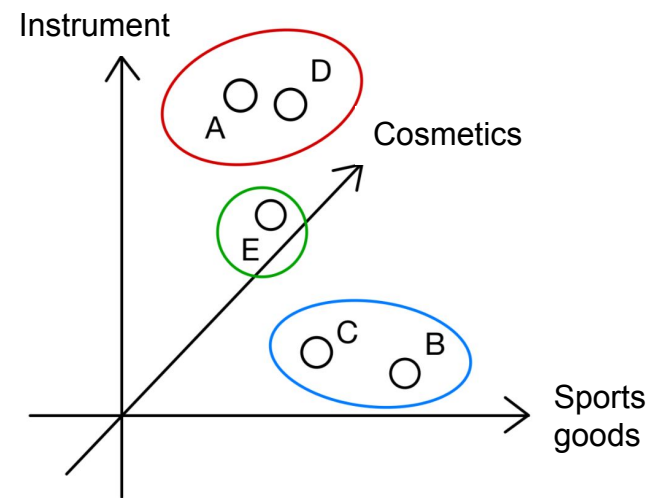
Customer segmentation

	age	sex	cosmetics	sports goods
A	18	F	20	1
B	21	M	2	8
C	50	F	12	0
D	24	F	25	2
E	48	F	14	1



Store segmentation

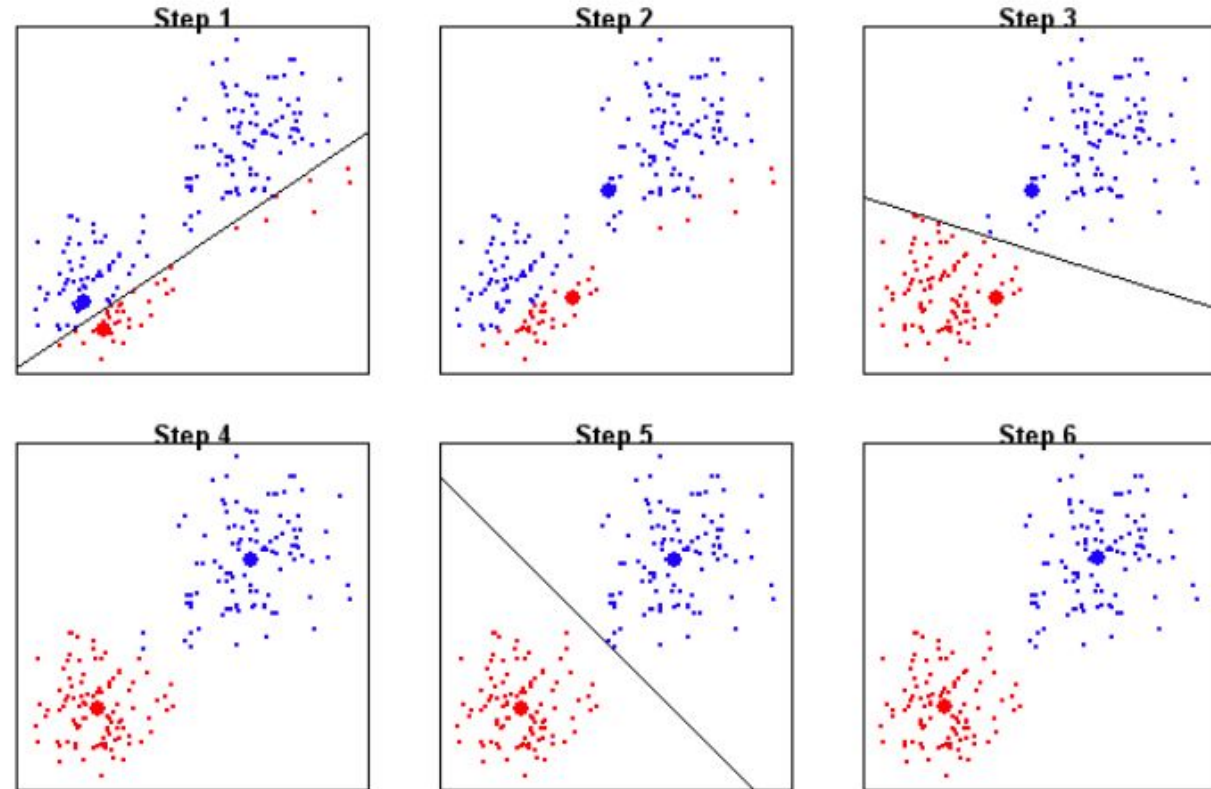
	cosmetics	sports goods	instrument	game
storeA	60	10	90	20
storeB	10	120	10	30
storeC	20	100	20	50
storeD	100	30	80	10
storeE	50	40	30	100



- A well-known clustering method

e.g. k-means clustering when $K=2$

1. Randomly assign a data point per cluster using its mean
2. Assign all samples to the cluster with the nearest mean
3. Update means of each cluster
4. Repeat 2-3 until the update is sufficiently small



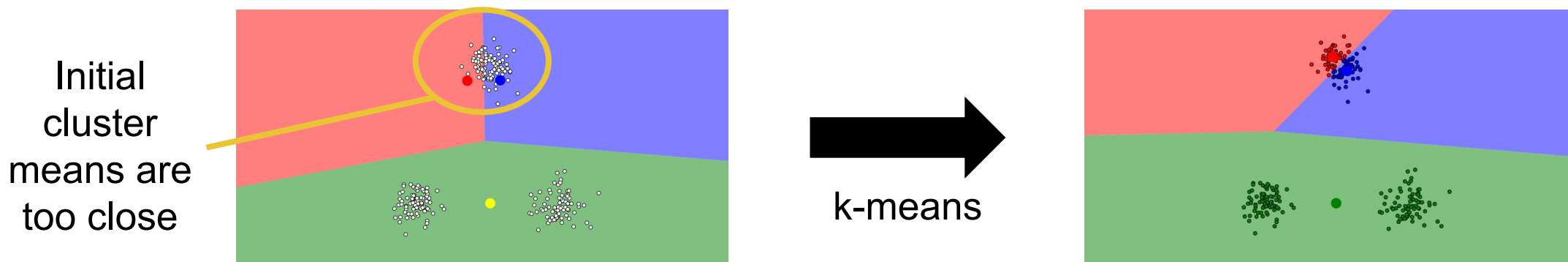
- Usually very fast in practice
- Clustering result may vary depending on initialization

Reference:

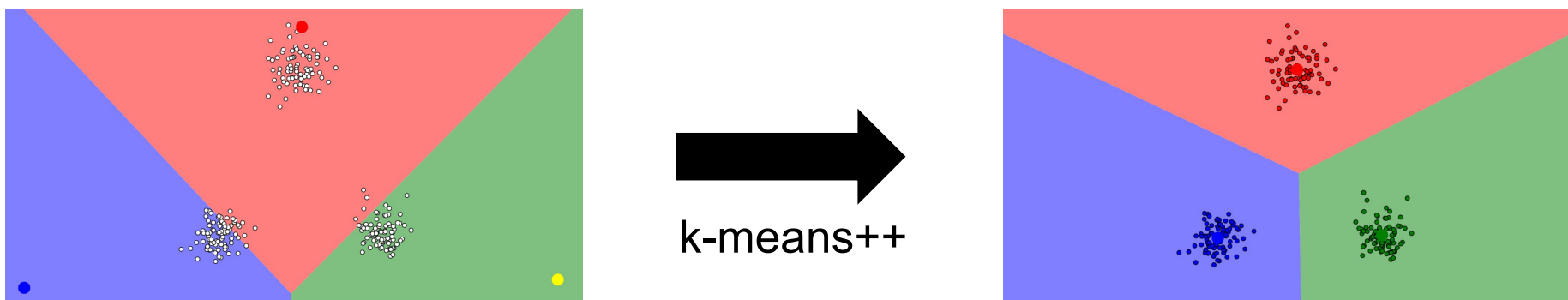
<https://people.eecs.berkeley.edu/~jrs/papers/machlearn.pdf>

k-means++

- Clustering result by k-means can vary depending on initial cluster means



- Solution by k-means++: initial means are placed as far apart as possible
 - When using clustering in practice, k-means++ is often used implicitly



Reference:

<https://www.naftaliharris.com/blog/visualizing-k-means-clustering/>

Deciding the Number of Clusters

- Number of clusters (k) is a hyperparameter for k-means and k-means++
 - But, you often do not know about the correct segmentation of the data

Methods:

1. Try all values, evaluate afterwards
 - **Elbow method**
 - **Silhouette analysis**
2. Automatically decide the optimal value
 - **Hierarchical clustering**
 - X-means, G-means

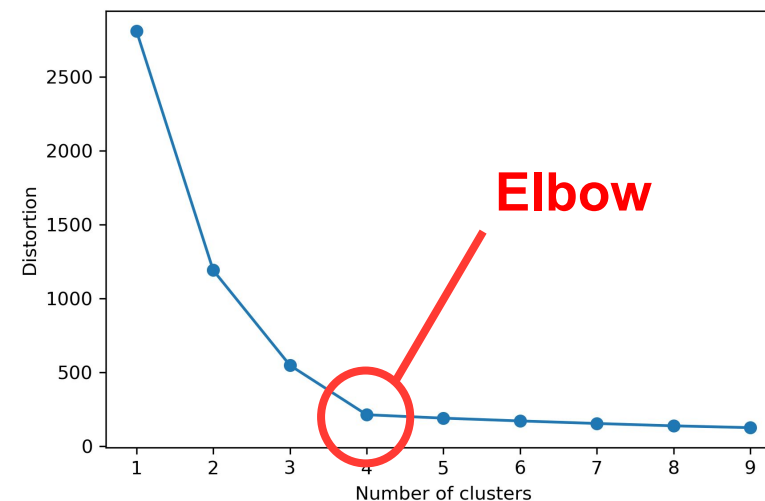
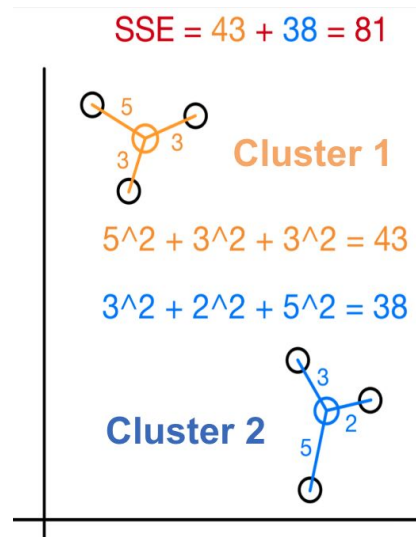
Elbow Method

- Evaluate how well each cluster are grouped by calculating “**distortion**”

- Usually use within-cluster sum of squared errors (SSE)
- Sum of squared difference between data point and cluster mean

- Increase number of clusters

-> each cluster will have less data points -> SSE will decrease

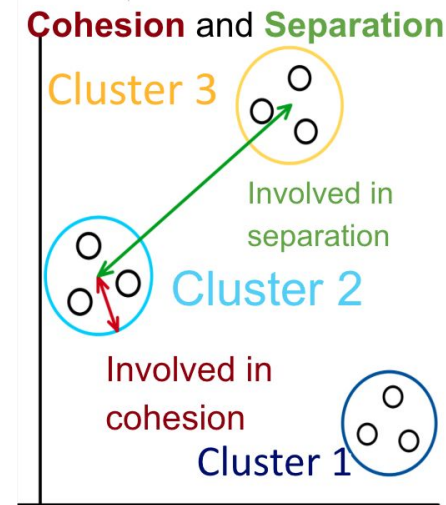


1. Calculate SSE for each k
2. Plot and find where adding clusters results in only a marginal decrease in SSE

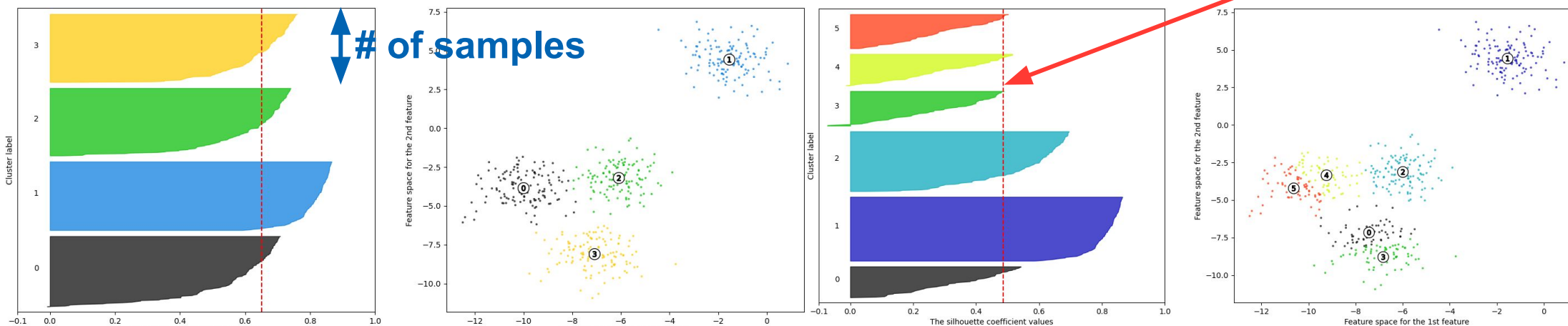
Silhouette Analysis

- Evaluate how well each cluster are grouped by calculating **silhouette coefficient** based on “**cohesion**” and “**separation**”

- Coefficient range: $[-1, 1]$
- Coefficient $\rightarrow 1$ = sample is far away from neighboring clusters
= good clustering



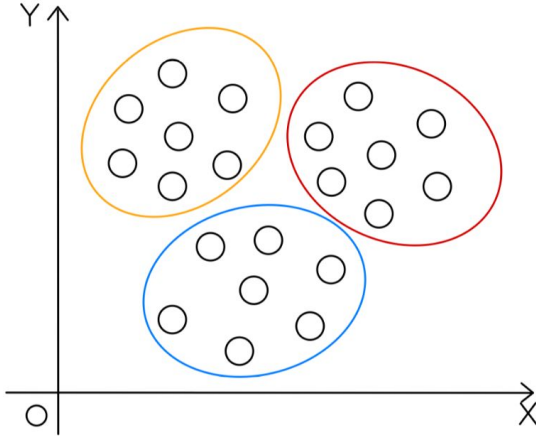
1. Calculate silhouette coefficient for each k
2. Plot and find where sufficient number of samples exceed average silhouette coefficient



Reference:

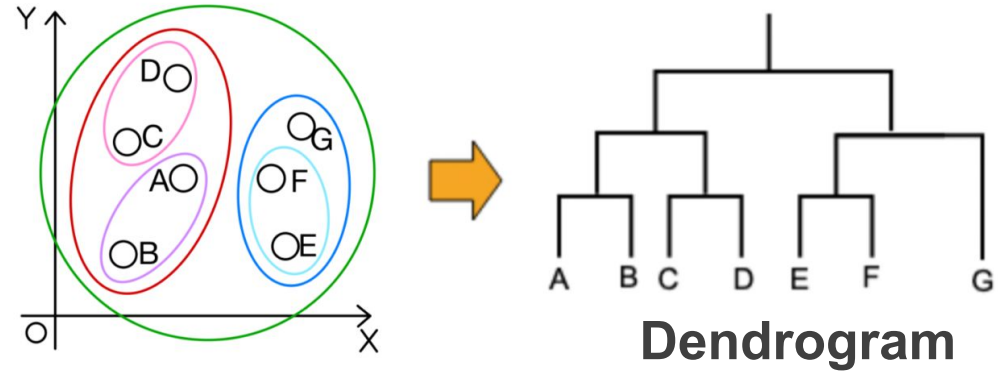
https://scikit-learn.org/1.5/auto_examples/cluster/plot_kmeans_silhouette_analysis.html

Non-hierarchical clustering



- No structure of clusters
- Number of clusters is decided in advance
- e.g.
 - **k-means**

Hierarchical clustering



- Hierarchical structure of clusters
- Sequentially merge nearby clusters (visualize using dendrogram)
- e.g.
 - **Ward method**

Interpreting Clustering Results

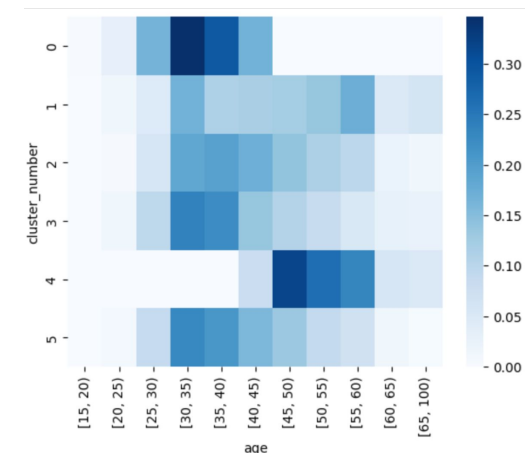
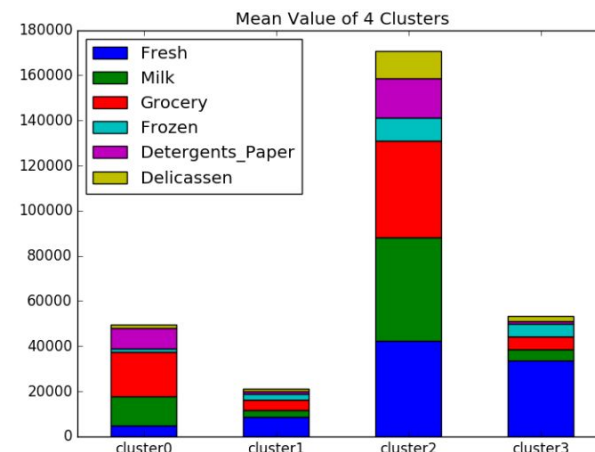


- Use groupby and view stats per cluster
 - Visualize mean, proportion, etc. of each feature
 - Use scatter plots, bar graphs, heatmaps, etc.

Note:

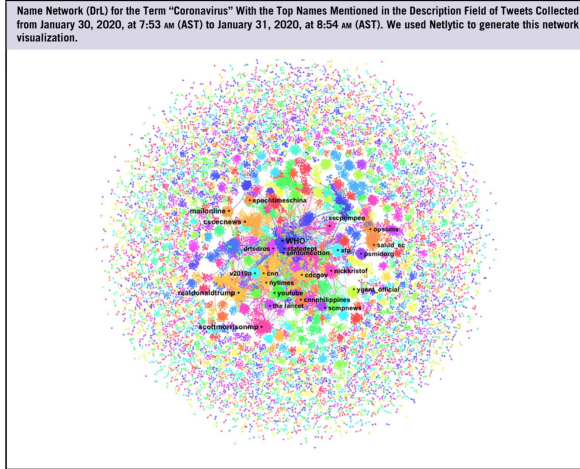
- Clustering is subjective
 - Non-hierarchical clustering: Number of clusters
 - Hierarchical clustering: Threshold of dendrogram
- Results are hard to interpret
 - Difficult to analyze what each cluster represents

-> **Analyze with trial-and-error, use feature selection, utilize domain knowledge**



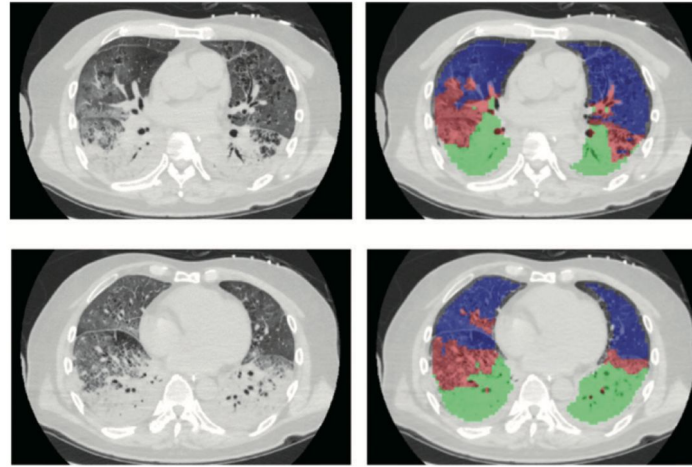
Examples of Clustering Applications

Natural Language Processing



- Based on social media posts or surveys, group customers with similar reactions
- > Customized marketing

Image Processing



- Segment areas of photo based on shading of each pixel
- > Use for medical analysis

Audio Processing



- Categorize music tracks based on genre
- > Create custom playlists of each genre

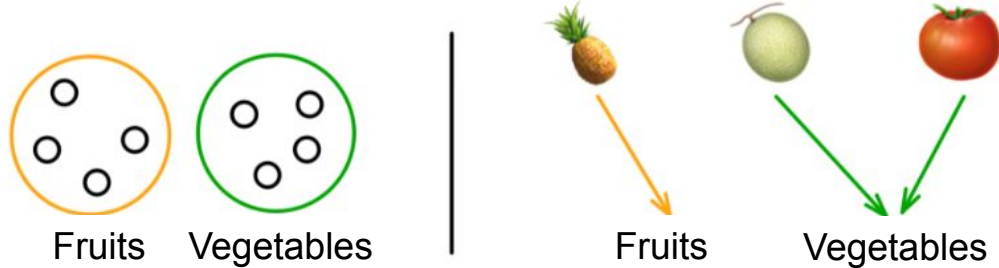
Reference:

<https://www.cambridge.org/core/journals/disaster-medicine-and-public-health-preparedness>

https://www.jstage.jst.go.jp/article/mit/39/1/39_13/article/-char/ja

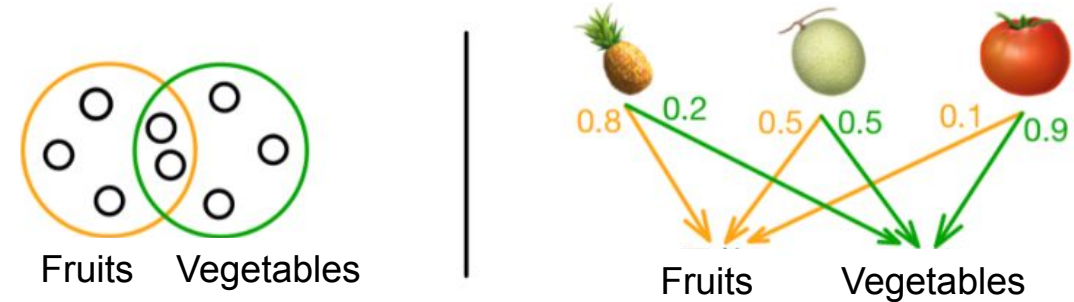
<https://zenn.dev/miqawari1253/articles/e6f7df001a355c>

Hard clustering



- Each data point is assigned to a single cluster
- e.g.
 - **k-means**

Soft clustering



- Each data point can belong to multiple clusters
- Calculate the probability of belonging to each cluster
- e.g.
 - **Mixture model**

- **Unsupervised learning** : Discovers patterns in the data
 - Contrary to supervised learning, it has no target values to predict
 - Example methods : **Clustering, Dimensionality Reduction**
- **Clustering** : Find similar patterns in the data and group them as clusters
 - Example methods : k-means, k-means++
 - How to determine clusters
 - Try all values, evaluate afterwards : Elbow method, Silhouette analysis
 - Automatically decide the optimal value : Hierarchical clustering
 - Applications : Natural Language Processing, Image Processing, Audio Processing

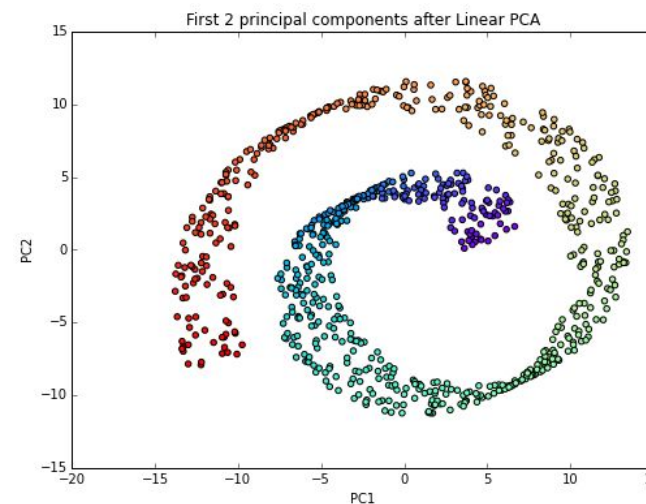
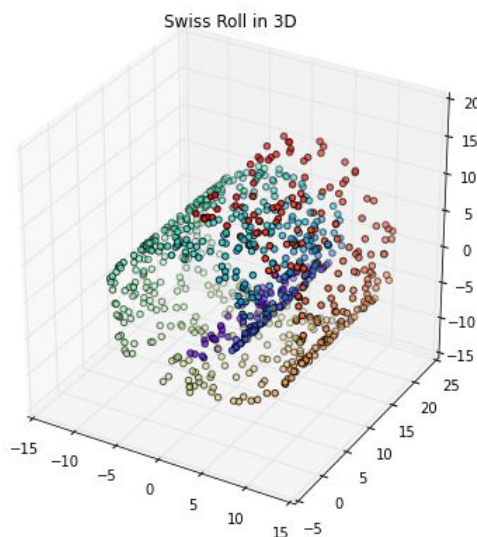
Dimensionality Reduction

Dimensionality Reduction

- High-dimensional data is hard to interpret
- Want to reduce dimensionality but preserve information (=structure/patterns of data) as much as possible

- Methods:

- **PCA**
- tSNE, UMAP
- etc.



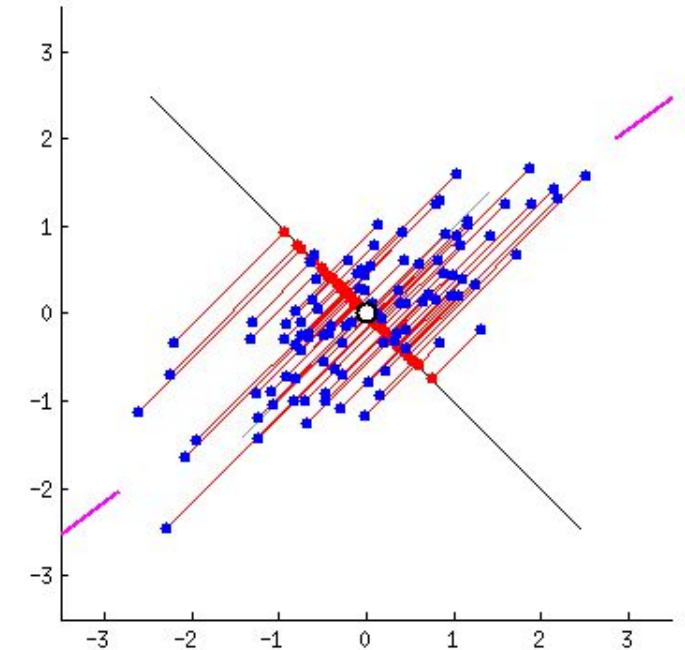
Reference:

https://sebastianraschka.com/Articles/2014_kernel_pca.html

Principal Component Analysis (PCA)



- Find a linear combination of attributes (**principal components, PC**) that best describes the data = variance is large
 - Find k more components in the same manner, but ensure each one is orthogonal to all previous ones
1. Calculate covariance matrix
 2. Calculate eigenvalues and eigenvectors of the covariance matrix
 3. Eigenvector with largest eigenvalue becomes the first PC
 4. Eigenvector with second largest eigenvalue becomes the second PC, ...



Reference:

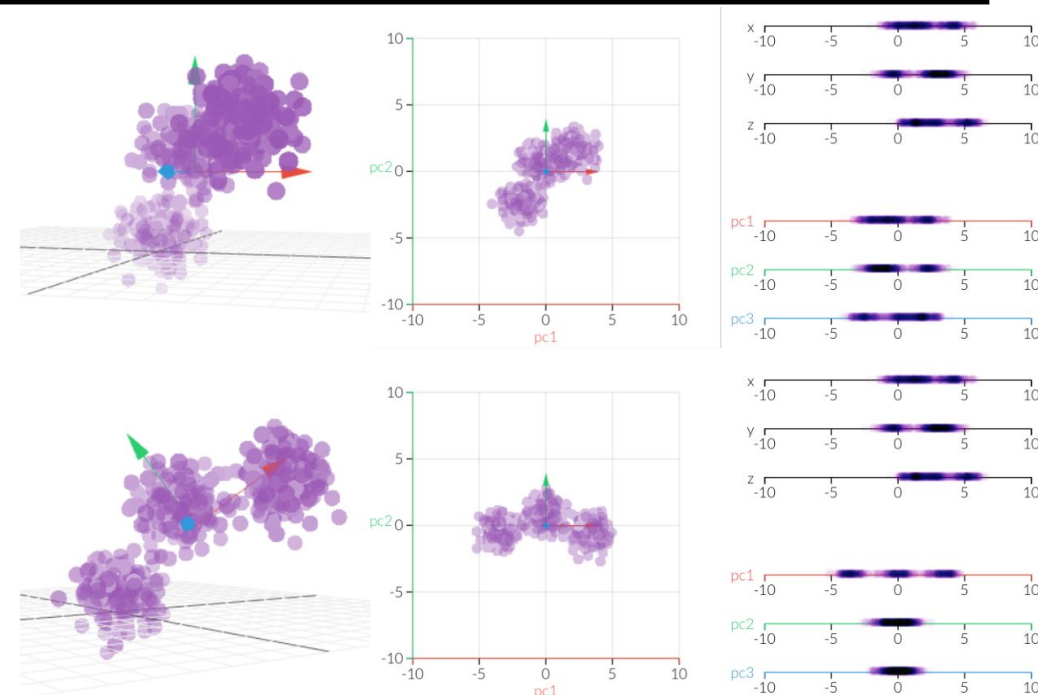
<https://stats.stackexchange.com/questions/2691/making-sense-of-principal-component-analysis-eigenvectors-eigenvalues/140579#140579>

Interpreting PCA Results

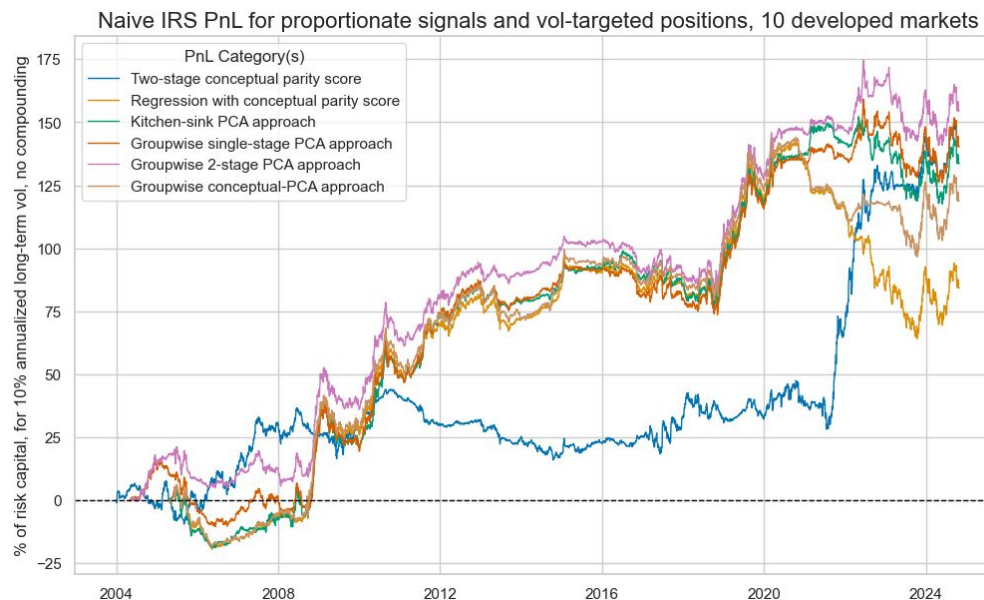
- Check **contribution ratio** (percentage of variability)
- Visualize your results to understand your data better

Note:

- Results are hard to interpret
 - Meaning of PCs is subjective
- Some information will be dropped
 - Dimensionality reduction = disregarding parts of the data
- Be careful with outliers
 - Outliers may distort direction and contribution rate of the PCs

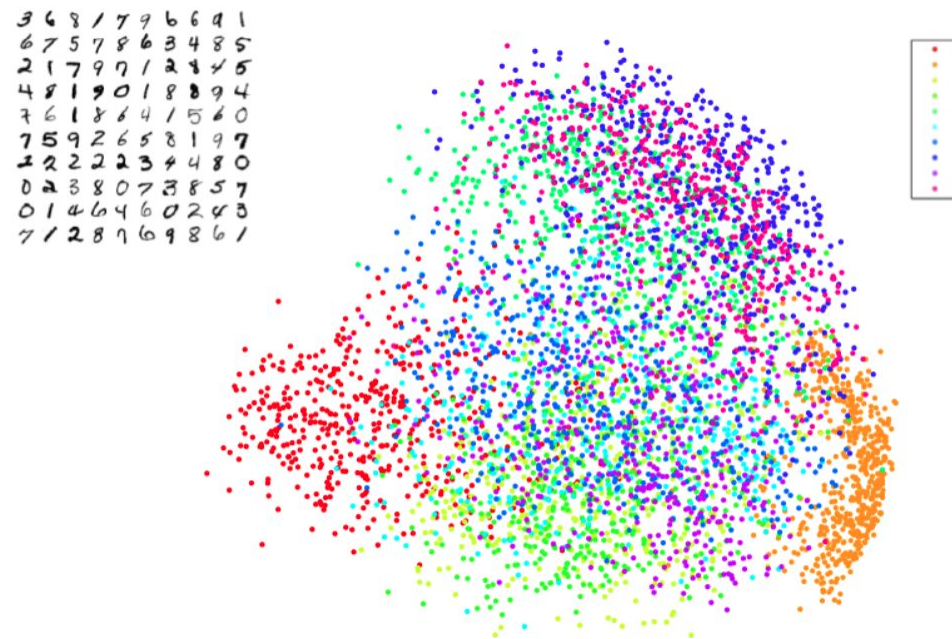


Macro Trading



- Apply PCA to develop macro trading signals
- > Predict market change to manage risk

Image Processing



- Apply PCA to images
- > Find similarities between images and understand the entire dataset better

Reference:

<https://macrosynergy.com/research/using-principal-components-to-construct-macro-trading-signals/>

<https://people.eecs.berkeley.edu/~jrs/papers/machlearn.pdf>

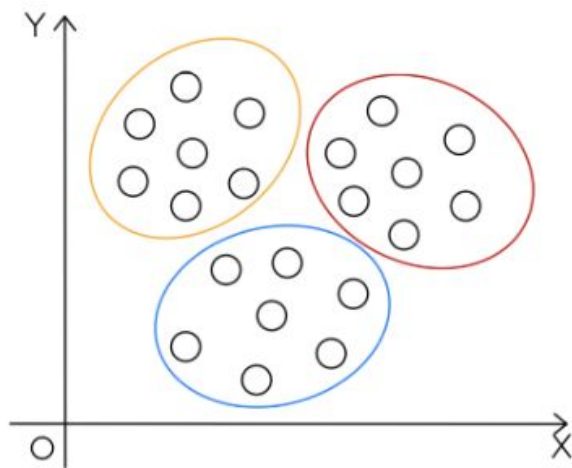
- **Dimensionality Reduction** : Reduce dimensionality of data while preserving information
 - Example methods : **PCA**, tSNE, UMAP
 - **PCA** : Find principal components that best describe the data
 - Check contribution ratio for interpretation
 - Applications : Macro Trading, Image Processing

Key Takeaways



- **Unsupervised learning** : Discovers patterns in the data
 - No target values to predict
 - **Clustering** : Find similar patterns in the data and group them as clusters (ex. k-means, k-means++)
 - **Dimensionality Reduction** : Reduce dimensionality of data while preserving information (ex. PCA)

Clustering



Dimensionality Reduction

